🧪 Projekt zespołowy – System testowania OOB modułu z PyPI

🎯 Cel projektu

Celem projektu jest zaprojektowanie i zrealizowanie uproszczonego systemu testowania OOB (Out Of The Box) wybranego modułu Pythonowego dostępnego w repozytorium PyPI.

Projekt ma charakter praktyczny. Jego zadaniem jest zapoznanie Was z:

- pracą zespołową nad repozytorium GitHub
- podstawami CI/CD (pipeline uruchamiana manualnie)
- testowaniem funkcjonalnym i wydajnościowym
- projektowaniem scenariuszy testów akceptacyjnych
- organizacją pracy projektowej

Projekt nie ma na celu stworzenia zaawansowanego systemu walidacyjnego. Ma być rozsądny objętościowo i wykonalny równolegle z innymi obowiązkami szkolnymi.

👥 Zespoły

Projekt realizowany jest w zespołach 3-osobowych.

Każdy zespół:

- zakłada repozytorium na GitHubie
- wybiera jeden moduł Pythonowy dostępny w PyPI
- pracuje nad projektem przez około 2,5 miesiąca

📦 Zakres podstawowy (obowiązkowy)

Projekt obejmuje testowanie najnowszej stabilnej wersji wybranego modułu z PyPI.

Wymagania minimalne:

1️⃣ Repozytorium

Repozytorium powinno zawierać:

- README z opisem projektu i strategii testowej
- opis podziału ról w zespole
- strukturę katalogów projektu
- dokument z opisem scenariuszy testów akceptacyjnych

---

2 Pipeline (GitHub Actions)

Pipeline powinna:

- być uruchamiana manualnie
- instalować wskazany moduł z PyPI
- uruchamiać testy jednostkowe biblioteki (jeśli są dostępne)
- uruchamiać Wasze testy funkcjonalne
- uruchamiać Wasze testy wydajnościowe
- wyświetlać czytelne podsumowanie wyników

Pipeline ma być prosta i przejrzysta. Nie wymagamy Dockerów, matrix buildów ani wielosystemowych konfiguracji.

---

3 Testy funkcjonalne

Każdy zespół przygotowuje:

- 3–5 testów funkcjonalnych

Testy powinny:

- sprawdzać realne użycie modułu
- mieć uzasadnienie (dlaczego akurat te scenariusze)
- być czytelne i dobrze nazwane

---

4 Testy wydajnościowe

Każdy zespół przygotowuje:

- 1–2 proste testy wydajnościowe

Nie wymagamy profesjonalnych benchmarków. Wystarczy:

- pomiar czasu wykonania wybranej operacji
- zapis wyniku do loga
- proste porównanie wyników

---

5 Scenariusze testów akceptacyjnych

Należy przygotować dokument zawierający:

- co najmniej 3 scenariusze testów akceptacyjnych
- opis celu testu
- opis oczekiwanego rezultatu
- kryterium zaliczenia

Nie muszą być one automatyzowane.

🏆 Opcja dla grup celujących w ocenę celującą

Dla zespołów ambitnych przewidziana jest możliwość rozszerzenia projektu:

🔧 Budowanie modułu z wybranego commita na GitHubie

Zamiast instalować wersję z PyPI, zespół może:

- pobrać wybrany commit z oficjalnego repozytorium GitHub modułu
- zbudować go lokalnie
- przetestować go przy użyciu swojej pipeline

Wymaga to:

- zrozumienia zależności
- rozwiązania ewentualnych problemów buildowych
- udokumentowania procesu

To rozszerzenie jest dobrowolne i przewidziane dla zespołów chcących celować w ocenę najwyższą.

📊 System oceniania

Moja ocena dotyczy projektu jako całości.

Projekt otrzymuje określoną liczbę punktów.

Następnie członkowie zespołu dzielą między sobą te punkty zgodnie z rzeczywistym wkładem pracy.

Podział punktów musi być:

- uzgodniony przez zespół
- opisany w dokumencie końcowym
- uzasadniony

Jeżeli podział będzie nieracjonalny lub niespójny z historią repozytorium, zastrzegam sobie prawo do jego weryfikacji.

📅 Harmonogram projektu

Projekt trwa około 2,5 miesiąca.

W trakcie realizacji przewidziane są 3 punkty kontrolne oraz jeden końcowy.

📌 Punkt kontrolny 1

Organizacja projektu

Oceniam:

- czy repozytorium zostało założone
- czy istnieje README z opisem celu projektu
- czy zespół przygotował harmonogram
- czy określono role w zespole
- czy ustalono kanały komunikacji
- czy zaplanowano wstępne scenariusze testowe

Na tym etapie kod może być jeszcze minimalny.

📌 Punkt kontrolny 2

Zarządzanie kodem

Oceniam:

- wykorzystanie Issues
- wykorzystanie Pull Requestów
- jakość commitów (czytelne opisy)
- podział zadań między członków zespołu
- czy występują code review
- czy repozytorium rozwija się w sposób uporządkowany

To etap pokazujący, czy projekt jest prowadzony profesjonalnie.

📌 Punkt kontrolny 3

Testowanie

Oceniam:

- czy testy funkcjonalne działają
- czy testy wydajnościowe działają
- czy pipeline się uruchamia
- czy testy są sensowne i uzasadnione
- czy istnieje raportowanie wyników
- czy scenariusze testów akceptacyjnych są logiczne

Nie oceniam ilości testów. Oceniam ich sens.

🏁 Ocena końcowa – Release

Oceniam:

- kompletność dokumentacji
- działającą pipeline
- przejrzystość repozytorium
- jakość prezentacji projektu
- rzetelność samooceny zespołu

Każdy zespół przygotowuje:

- krótką prezentację projektu
- omówienie problemów, które napotkał
- refleksję: co by zrobili inaczej

🧭 Czego ten projekt ma Was nauczyć

Nie chodzi o perfekcyjne DevOps-owanie. Projekt też nie ma być za bardzo pracochłonny.

W realizacji tego projektu chodzi przede wszystkim o:

- odpowiedzialność zespołową
- organizację pracy
- realne testowanie gotowego oprogramowania
- rozróżnianie rodzajów testów
- czytelność kodu i dokumentacji

Realizacja ma być ambitna, ale i też rozsądna.

Lista propozycji modułów do testowania (dwie grupy nie mogą realizować testów OOB tego samego modułu, więc kto pierwszy ten lepszy):

1. **NumPy** (Numerical Python) – fundament obliczeń na tablicach wielowymiarowych i macierzach, baza pod sporą część ekosystemu data-science. Poziom: *średni*.
2. **Pandas** – praca na danych tabelarycznych (DataFrame), filtrowanie, agregacje, łączenia, czyszczenie danych. Poziom: *średni / trudniejszy*.
3. **Pillow** (PIL Fork) – przetwarzanie obrazów: formaty plików, konwersje, filtry, transformacje. Poziom: *łatwy / średni*.
4. **Matplotlib** – klasyczna biblioteka do wykresów 2D/3D; dobre pole do testów generowania artefaktów (pliki PNG/SVG). Poziom: *średni*.
5. **SciPy** – obliczenia naukowe (optymalizacja, statystyka, sygnały, algorytmy numeryczne) w oparciu o NumPy. Poziom: *trudniejszy*.
6. **scikit-learn – ML** w stylu „klasycznym": klasyfikacja, regresja, clustering, pipeline'y, metryki. Poziom: *trudniejszy*.
7. **scikit-image** – przetwarzanie obrazów (segmentacja, filtry, morfologia, miary jakości obrazu), świetne do testów funkcjonalnych. Poziom: *średni / trudniejszy*.
8. **OpenCV-Python** (cv2) – potężna biblioteka wizji komputerowej: filtry, detekcje, transformacje, wideo. Poziom: *trudniejszy*.
9. **imageio** – prosty odczyt/zapis obrazów i sekwencji (GIF, TIFF, itp.), wygodne do testów I/O i konwersji formatów. Poziom: *łatwy / średni*.
10. **seaborn** – wizualizacje statystyczne „na Matplotlib", szybkie wykresy dla danych tabelarycznych. Poziom: *łatwy / średni*.
11. **Plotly** – interaktywne wykresy (HTML), sporo scenariuszy eksportu i walidacji danych wejściowych. Poziom: *średni*.
12. **Bokeh** – interaktywne wizualizacje w przeglądarce, podobnie jak Plotly, ale inny styl API. Poziom: *średni*.
13. **Altair** – deklaratywne tworzenie wykresów (Vega-Lite), fajne do testów „czy z danych powstaje poprawna specyfikacja". Poziom: *średni*.
14. **statsmodels** – statystyka i modele (regresje, testy statystyczne, szeregi czasowe), dobre do testów stabilności wyników. Poziom: *trudniejszy*.
15. **SymPy** – matematyka symboliczna (upraszczanie, całki, pochodne, równania), ciekawe testy poprawności i regresji. Poziom: *średni / trudniejszy*.
16. **NetworkX** – grafy i sieci (węzły, krawędzie, miary centralności, ścieżki), świetne do testów algorytmicznych. Poziom: *średni*.
17. **xarray** – dane wielowymiarowe z etykietami (często klimat meteo/ocean), naturalne rozszerzenie podejścia „NumPy + metadane". Poziom: *trudniejszy*.
18. **Dask** – obliczenia równoległe i „większe niż RAM" na strukturach podobnych do NumPy/Pandas. Poziom: *trudniejszy*.
19. **PyArrow** – kolumnowy format danych (Arrow), integracje i wydajne przetwarzanie; dobre do testów wydajności I/O. Poziom: *trudniejszy*.
20. **Polars** – szybkie DataFrame'y (alternatywa dla Pandas), wdzięczne do porównań wydajności i prostych testów funkcjonalnych. Poziom: *średni / trudniejszy*.

Tech leaders

- Baczyński Jakub
- Dąbrowski Adam
- Kamiński Maciej
- Lisiecki Gabriel
- Mierzwa Daniel
- Miłuch Hubert
- Rampalski Jan
- Schwarz Leon
- Ślugajska Alicja
- Zając Adrian